

FIREFOX AGENT

Technical Product & Architecture Specification

Claude-style sidebar + autonomous browser agent + NVIDIA NIM model router

Snapshot: 22 September 2026

Core idea

A Firefox-first agent workspace where the user connects an NVIDIA API key, discovers available models automatically, sees only the controls each model supports, and then gives the agent browser, search, file, coding and terminal capabilities through a guided sidebar.

Contents

- 1. Product vision and scope
- 2. User experience and onboarding
- 3. System architecture
- 4. NVIDIA NIM model discovery
- 5. Dynamic parameter system
- 6. Agent orchestration
- 7. Browser automation: Browser Use + JEV + Firefox Bridge
- 8. OpenCLI and terminal/coding capabilities
- 9. Skills system for documents and search
- 10. Neko sandbox mode
- 11. Security and permission model
- 12. Project structure
- 13. Example request lifecycle
- 14. MVP roadmap
- 15. Technical risks and mitigations
- 16. Definition of done
- 17. Current external references

1. Product vision and scope

The product is a Firefox extension with a persistent sidebar that feels closer to a modern AI workspace than a conventional browser add-on. The sidebar is the user-facing layer; the heavy agent runtime stays outside the extension in a local host/service.

The target experience is: connect NVIDIA once -> discover models -> select a model or profile -> adjust model-specific controls visually -> grant capabilities -> run tasks. The user should not need to understand API payloads, tool schemas or agent internals.

Design principle

Models are data, not branches in the code. The application should not contain a growing list of "if model == X" conditions. It should discover a model, normalize its capabilities, discover/validate its accepted parameters, and render the UI from a schema.

Primary capabilities

- Chat and task execution from a Firefox sidebar.
- Web search, page understanding, extraction and multi-step browser actions.
- Optional use of the user's current browser context or an isolated sandbox browser.
- Coding workflows using terminal/bash and OpenCLI-style browser/desktop primitives.
- Skills for PDF, DOCX, XLSX, web research, search, Git and other reusable operations.
- Dynamic model selector driven by NVIDIA metadata and runtime validation.
- Dynamic parameter controls driven by a normalized parameter schema.
- Permission and confirmation gates for sensitive browser, filesystem and terminal actions.

2. User experience and onboarding

The sidebar should guide rather than expose configuration complexity. Advanced settings exist, but the default surface stays simple and conversational.

2.1 Sidebar

```
+-----+
| ♦ Agent |
+-----+
| What do you want to do? |
|                           |
| Search the documentation and create the |
| Node.js project.         |
|                           |
| [ attachment ] [ web ]   [ Send ] |
+-----+
| Model: Kimi K3 ▼         |
| Multimodal · Reasoning · Tools |
|                           |
| ⚙ Parameters 🔒 Permissions |
+-----+
```

2.2 Guided onboarding

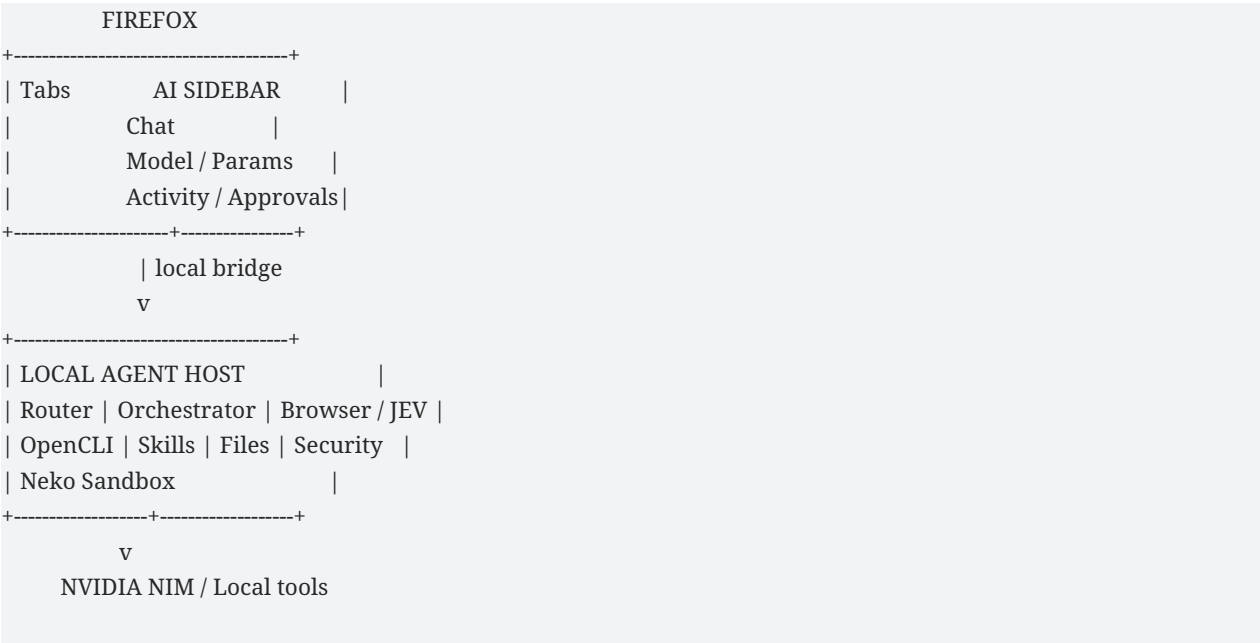
- 1. Connect NVIDIA API key.
- 2. Test connectivity and discover model catalogue.
- 3. Choose a default model or an agent profile.
- 4. Detect supported capabilities (reasoning, tools, vision, audio, video, etc.).
- 5. Render only the relevant input controls.
- 6. Choose permissions for browser, terminal and files.
- 7. Start the first task with a preflight summary.

2.3 Simple vs Advanced configuration

Surface	Simple mode	Advanced mode
Reasoning	Fast / Balanced / Deep	Raw reasoning_effort or model-specific field
Creativity	Slider / preset	temperature, top_p
Output	Context-aware default	max_tokens
Determinism	Auto	seed
Streaming	On/Off	stream
Advanced	Hidden by default	frequency_penalty, presence_penalty, stop, custom fields

3. System architecture

The extension should remain lightweight. Browser automation, model routing, skills and terminal operations run in a local agent host so they can use Python/Node, native processes and Docker when required.



3.1 Components

Component	Responsibility
Firefox Extension	Sidebar UI, content/context capture, user interaction, approvals, local host bridge.
Local Agent Host	Long-running local process for orchestration, model access, skills, files, terminal and browser control.
Model Router	Provider-neutral interface that selects model, profile, capabilities and parameter schema.
Agent Orchestrator	Maintains task state, tool loop, observations, retries, checkpoints and final response.
Browser Layer	Firefox bridge for live browser; Browser Use for general agent control; JEV for fast indexed element actions.
OpenCLI Layer	CLI/browser/desktop primitives and optional terminal workflows.
Skills Layer	Load-on-demand procedural instructions and adapters for documents, search and development tasks.
Sandbox Layer	Neko-based isolated browser sessions when the user opts out of the live Firefox session.
Security Layer	Secrets, permissions, confirmations, audit log and kill switch.

4. NVIDIA NIM model discovery

The model layer must treat NVIDIA's catalogue as dynamic. On 22 September 2026 the public NVIDIA model catalogue lists 99 models, with filters for free endpoints, partner endpoints and downloadable models. The catalogue currently includes, among others, GLM-5.3, GLM-5.3 Flash and Kimi K3. [Sources: NVIDIA Models catalogue]

4.1 Discovery pipeline

1. Fetch/refresh catalogue metadata.
2. Normalize model identifiers, publisher, endpoint availability and modalities.
3. Fetch model-specific documentation/model card when available.
4. Extract capabilities and parameter definitions.
5. Run a lightweight compatibility probe when the provider permits it.
6. Store a versioned local registry with timestamp and source URL.
7. Expose only models that pass the validation rules.

4.2 Important distinction: catalogue vs runtime schema

The build.nvidia.com catalogue is excellent for discovery, but it should not be treated as the sole source of truth for every request parameter. The implementation should combine catalogue metadata, official

model documentation and runtime validation. This prevents a UI change on the catalogue site from silently breaking inference.

4.3 Registry example

```
{
  "id": "moonshotai/kimi-k3",
  "displayName": "Kimi K3",
  "provider": "nvidia",
  "endpoint": "https://integrate.api.nvidia.com/v1",
  "capabilities": {
    "text": true,
    "vision": true,
    "reasoning": true,
    "tool_calling": true,
    "coding": true,
    "agentic": true
  },
  "parametersSchema": "kimi-k3-v1",
  "source": "build.nvidia.com / model card",
  "discoveredAt": "2026-09-22T..."
}
```

5. Dynamic parameter system

This is a core feature. Different models expose different parameter surfaces. The UI therefore must be generated from a schema rather than hardcoded per model.

5.1 Example model difference

Parameter	GLM-5.3 example	Kimi K3 example
stream	Available	Available
max_tokens	Available	Available
temperature	Available	Available
top_p	Available	Not necessarily exposed in the same playground surface
frequency_penalty	Available in the cited playground example	Not necessarily exposed in the same playground surface
presence_penalty	Available in the cited playground example	Not necessarily exposed in the same playground surface
stop	Available in the cited playground example	Not necessarily exposed in the same playground surface
reasoning_effort	May be model-specific / runtime-dependent	low / high / max
seed	Available in the cited playground example	Available

The exact availability must be discovered/validated rather than assumed from this table. NVIDIA's current catalogue/model documentation shows reasoning and tool use as model capabilities for several relevant models, and the request layer should preserve model-specific controls. [Sources: NVIDIA Models catalogue; official model pages]

5.2 JSON Schema-like definition

```
{
  "temperature": { "type": "number", "min": 0, "max": 2, "default": 1.0 },
  "max_tokens": { "type": "integer", "min": 1, "max": 131072 },
  "stream": { "type": "boolean", "default": true },
  "reasoning_effort": {
    "type": "enum",
    "values": ["low", "high", "max"],
    "default": "high"
  },
  "seed": { "type": "integer", "optional": true }
}
```

5.3 UI renderer

The sidebar renders widgets from schema types: boolean -> toggle; number -> slider or numeric field; integer -> numeric input; enum -> segmented control/dropdown; string -> text field; array -> multi-select; object -> nested advanced section. Optional fields appear only when the schema says they exist.

5.4 Presets / profiles

Profile	Purpose
Fast	Low-latency defaults; lower reasoning budget where supported.
Balanced	General-purpose browser/chat settings.
Deep reasoning	Higher reasoning effort for complex research, planning or coding.
Browser agent	Conservative interaction settings, strong tool use and explicit approvals.
Coding agent	Reasoning-heavy, low-temperature defaults and terminal/file safeguards.
Custom	Expose all validated parameters supported by the selected model.

6. Agent orchestration

The selected model is not the agent by itself. The agent combines a language model with tools, observations, memory/state, skills and permissions.

```
User goal
-> Task planner
-> Skill router
-> Capability check
```

```
-> Tool selection
-> Observation
-> Model decision
-> Tool execution
-> Observation ...
-> Completion / approval request
```

6.1 Tool classes

- Browser tools: navigate, inspect DOM, click, type, select, scroll, screenshot, extract.
- Search tools: web search, page retrieval, result ranking, citation capture.
- File tools: read/write/list files; parse PDF/DOCX/XLSX; produce artifacts.
- Terminal tools: execute commands, inspect output, install dependencies, run tests.
- Desktop/CLI tools: OpenCLI adapters for supported websites or Electron apps.
- Approval tools: request user confirmation before sensitive actions.

7. Browser automation: Browser Use + JEV + Firefox Bridge

Browser Use is a general browser-agent framework; its current project exposes an MCP server for browser control. JEV Ultrafast is designed around a dynamic, indexed action space where the agent chooses an operation and element, calling a small LLM for text generation only when TYPE_TEXT is needed. [Sources: Browser Use, JEV Ultrafast]

7.1 Division of labour

- Browser Use = general multi-step agent execution and resilient browser control.
- JEV = fast indexed element/action loop for repetitive or latency-sensitive interactions.
- Firefox Bridge = native Firefox-specific bridge that exposes the current tab/session safely to the local host.
- Neko = optional isolated browser execution mode.

Firefox-specific constraint

OpenCLI's current Browser Bridge documentation is Chrome-based and explicitly describes a Chrome extension plus micro-daemon. Therefore the Firefox product should implement its own WebExtension bridge, borrowing the conceptual contract rather than assuming OpenCLI is Firefox-native. [Sources: OpenCLI Browser Bridge docs]

8. OpenCLI and terminal/coding capabilities

OpenCLI provides browser/desktop automation primitives and a bridge model intended for AI agents; its current docs also describe reusable skills and website adapters. The project currently requires Chrome for its documented browser mode, so Firefox integration is a separate bridge problem. [Sources: OpenCLI Browser Bridge / Getting Started]

8.1 Command permission policy

Class	Examples	Default
Read-only	pwd, ls, git status, inspect files	Allow

Reversible write	create/edit source files, npm install	Allow with audit
Network/system change	service control, firewall changes	Ask first
Destructive	rm -rf, disk formatting, credential deletion	Always confirm
Secrets	read API keys, browser tokens, password stores	Block / explicit scoped permission

9. Skills system for documents and search

Skills are small, load-on-demand procedural packages. The router should select only the skills relevant to the current task, preventing prompt bloat and making the system extensible.

```
skills/
  browser/
    search/
    research/
    extraction/
  documents/
    pdf/
    docx/
    xlsx/
    pptx/
  coding/
    git/
    python/
    node/
    shell/
  web/
    citations/
    source-evaluation/
```

9.1 Skill lifecycle

1. Classify task.
2. Select minimal skill set.
3. Load each skill manifest/instructions.
4. Expose tools required by those skills.
5. Execute with permissions.
6. Unload or cache skill state depending on reuse.

10. Neko sandbox mode

Neko is a self-hosted virtual browser that runs in Docker and uses WebRTC, making it suitable as an optional isolated execution target. [Source: Neko repository]

```
LIVE MODE
User Firefox -> Firefox Bridge -> Agent
```

SANDBOX MODE

Sidebar -> Neko session manager -> isolated browser -> Agent

The sidebar should make the distinction visible: "Use my current browser" vs "Open isolated browser". Sandbox mode is especially useful for clean sessions, reproducibility, or tasks where the user does not want the agent to touch their existing browser state.

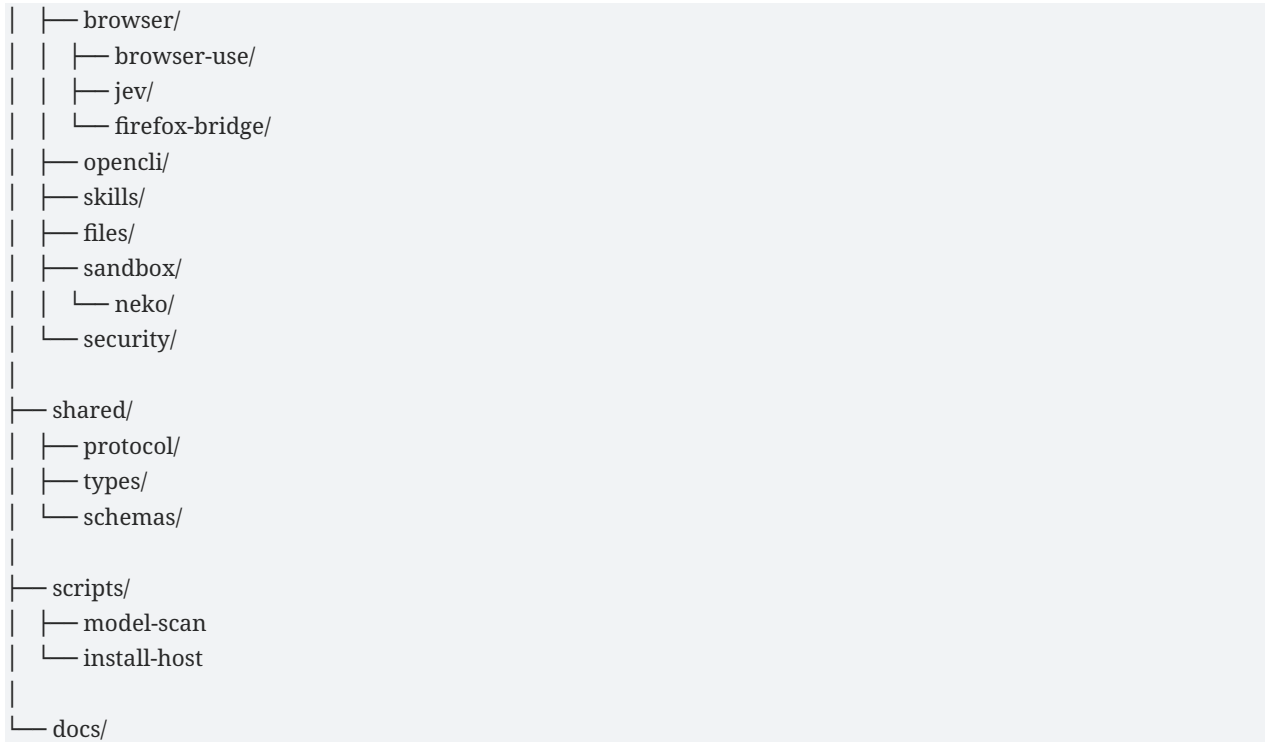
11. Security and permission model

Because the product can browse, execute code and manipulate files, security is a first-class subsystem rather than an afterthought.

- API keys stored outside page-accessible content scripts; use the local agent host/OS credential storage where practical.
- Never expose provider secrets to arbitrary webpages.
- Per-tool permission scopes: browser, terminal, filesystem, network, clipboard, downloads.
- Explicit confirmation for destructive or irreversible actions.
- Visible activity timeline showing what the agent is doing and why.
- Stop button / emergency kill switch that terminates active tool loops.
- Host allowlist and path restrictions for terminal/filesystem actions.
- Audit log for commands, file writes, downloads and high-risk browser actions.

12. Project structure

```
firefox-agent/  
├── extension/  
│   ├── sidebar/  
│   │   ├── chat/  
│   │   ├── onboarding/  
│   │   ├── model-selector/  
│   │   ├── parameter-editor/  
│   │   ├── permissions/  
│   │   └── activity/  
│   ├── background/  
│   ├── content/  
│   ├── manifest.json  
│   └── icons/  
└── host/  
    ├── gateway/  
    ├── orchestrator/  
    ├── model-router/  
    │   ├── discovery/  
    │   ├── registry/  
    │   ├── schemas/  
    │   ├── validation/  
    └── providers/
```



13. Example request lifecycle

Example: "Search the official Docker documentation, compare the installation steps, create a starter Node.js project, and run the test."

- 1. Sidebar sends user task to local agent host.
- 2. Task classifier selects web research + browser + coding + shell skills.
- 3. Model router verifies selected model has tool calling and required modalities.
- 4. Agent opens documentation in browser and gathers source-backed facts.
- 5. Agent creates files using terminal/file tools.
- 6. Agent runs tests; if a destructive command is proposed, approval is requested.
- 7. Agent returns a concise summary with files changed and sources consulted.

14. MVP roadmap

Phase	Deliverable
MVP-0	Firefox sidebar + local host + NVIDIA API key + one manually selected model.
MVP-1	Dynamic model discovery + normalized registry + basic parameter renderer.
MVP-2	Firefox Bridge + browser-use + page inspection + click/type/scroll/extract.
MVP-3	Skills engine + web search + PDF/DOCX/XLSX pipelines.

MVP-4	JEV fast-action path + OpenCLI terminal/browser adapter.
MVP-5	Neko sandbox mode + permission center + audit log.
MVP-6	Polish: profiles, streaming UX, retries, observability, packaging and Firefox distribution.

15. Technical risks and mitigations

Risk	Mitigation
Provider metadata drift	Do not rely on HTML scraping alone; combine catalogue, model docs and runtime probes.
Model-specific parameters	Use schema-driven UI and request validation; reject unsupported fields.
Browser compatibility	Create a Firefox-native bridge; keep Browser Use/JEV/OpenCLI behind internal interfaces.
Long-running agent loops	Use checkpoints, max steps, timeout policies and a visible stop button.
Prompt/tool misuse	Strict tool schemas, permission gates and path/network restrictions.
Secrets exposure	Keep API keys in local host/secure storage, never in page context.
Changing open-source APIs	Pin versions initially and add integration tests for each adapter.
UX overload	Simple defaults + advanced section; guided onboarding and clear activity timeline.

16. Definition of done

- User can install the Firefox extension and open the persistent sidebar.
- User can enter an NVIDIA API key and test connectivity without exposing it to page scripts.
- Application discovers and lists current supported models from NVIDIA metadata.
- Each model has normalized capabilities and a validated parameter schema.
- Sidebar hides unsupported parameters and renders model-specific controls automatically.
- User can switch between simple presets and advanced parameters.
- Agent can browse through a Firefox bridge and use browser-use.
- Agent can choose a fast-action path with JEV where applicable.
- Agent can call skills for search and document workflows.
- Agent can use terminal/file operations through a guarded local host.
- User can choose live browser or Neko sandbox mode.
- Every high-risk action requires explicit confirmation and is visible in the activity log.
- Integration tests cover model discovery, parameter validation, browser bridge and terminal permissions.

17. Current external references

NVIDIA NIM model catalogue - <https://build.nvidia.com/models>

Current model discovery and catalogue metadata; snapshot checked 22 Sep 2026.

Browser Use repository - <https://github.com/browser-use/browser-use>

Browser agent framework and current MCP server packaging.

JEV Ultrafast repository - <https://github.com/browser-use/jev-ultrafast>

Indexed browser-action agent designed for fast, low-cost interaction.

OpenCLI Browser Bridge - <https://github.com/jackwener/OpenCLI/blob/main/docs/guide/browser-bridge.md>

Chrome extension + micro-daemon browser bridge architecture.

OpenCLI Getting Started - <https://github.com/jackwener/opencli/blob/main/docs/guide/getting-started.md>

Browser/desktop automation and AI-agent-oriented CLI primitives.

Neko repository - <https://github.com/m1k1o/neko>

Self-hosted virtual browser, Docker and WebRTC.

NVIDIA model docs - <https://docs.nvidia.com/nim/large-language-models/latest/>

NIM inference and API documentation.